

Exercise Session 5: Neural Networks, CNNs, Regularization, Model Selection, Loss Functions

Automatic Image Analysis

Exercise Session 5

Core modeling ideas for modern visual pipelines: feed-forward neural networks, convolutional structure, regularization, model selection, and loss design.

- What the model computes.
- How to keep it from overfitting.
- How to choose and evaluate a training setup.

Computer Vision and Remote
Sensing Group, TU Berlin
Summer Semester 2026

Roadmap

Model concepts

- Feed-forward neural networks.
- Nonlinearity and representation learning.
- CNN structure and feature hierarchies.

Training concepts

- Loss functions.
- Regularization.
- Model selection and validation.

Why neural networks at all?

Main idea

A neural network learns a parametric mapping from input to output by composing many simple transformations.

- Linear models are often too limited for image data.
- Neural networks can represent nonlinear decision boundaries.
- Deeper layers can build progressively more abstract features.

Course view

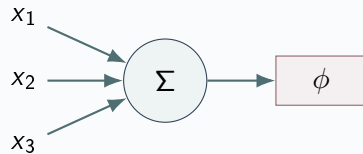
The classical pipeline engineers features first. Neural networks try to learn the useful representation directly from data.

A single neuron

Computation

$$z = w^{\top} x + b, \quad a = \phi(z)$$

- x : input vector
- w : weights
- b : bias
- ϕ : activation function



Why nonlinearity matters

Without activation functions

A stack of linear layers is still only one linear mapping.

Typical activations

- ReLU: $\max(0, z)$
- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$
- Tanh: maps to $[-1, 1]$

Practical point

ReLU is simple, fast, and usually the default in modern CNNs.

From neurons to layers

Layer notation

For a hidden layer,

$$h = \phi(Wx + b).$$

For a full network,

$$f(x) = W_L \phi(W_{L-1} \phi(\cdots \phi(W_1 x + b_1) \cdots) + b_{L-1}) + b_L.$$

- Early layers detect simple patterns.
- Middle layers combine them into larger structures.
- Final layers support classification or regression decisions.

Neural network training

- ➊ Run a forward pass to compute predictions.
- ➋ Compute the loss comparing predictions and targets.
- ➌ Backpropagate gradients through the network.
- ➍ Update parameters with an optimizer such as SGD or Adam.
- ➎ Repeat over mini-batches for many epochs.

Key point

Training is optimization of a loss, not direct optimization of accuracy.

Where CNNs fit in

Fully connected network

Treats the image as an unordered long vector.

CNN

Preserves spatial structure and learns local visual filters.

Hierarchy

- Early layers: edges, corners, textures.
- Middle layers: motifs, parts, repeated structures.
- Late layers: object- or class-relevant patterns.

CNN building blocks

Convolution

Local filtering with shared weights.

Activation

Usually ReLU after convolution.

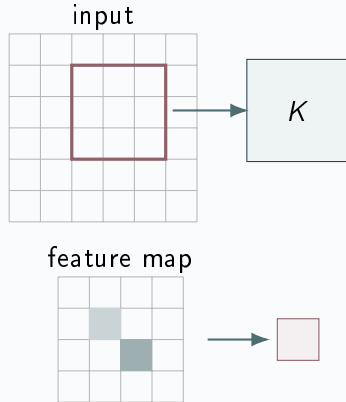
Pooling

Reduces spatial resolution and computation.

Often added

Batch normalization, residual shortcuts, global average pooling, and a final linear classification head.

How a convolution layer works



Computation

For output channel k ,

$$y_{ij}^{(k)} = \sum_{u,v,c} K_{uvc}^{(k)} x_{i+u,j+v,c} + b^{(k)}.$$

Meaning

The kernel slides across the image and computes a local weighted sum at each spatial location.

Stride, padding, and output size

Output size

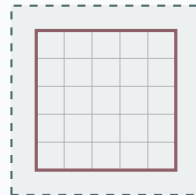
For input width W , kernel size F , padding P , and stride S :

$$W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1.$$

Interpretation

Stride controls downsampling. Padding controls border handling and whether spatial size is preserved.

padding around input



Typical settings

Small kernels such as 3×3 with stride 1 and moderate padding are standard in many CNNs.

What is a loss function?

Definition

A loss function assigns a penalty to a prediction relative to the target.

- The optimizer sees the loss, not your research story.
- A poor loss can push the model toward the wrong behavior.
- The chosen loss must match the task and label structure.

Cross-entropy for classification

Binary case

Let the model output a logit $z \in \mathbb{R}$. Convert it to a probability with

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

For target $y \in \{0, 1\}$, binary cross-entropy is

$$\mathcal{L}_{\text{BCE}}(y, p) = -y \log p - (1 - y) \log(1 - p).$$

This penalizes the negative log-likelihood of the correct binary label.

Multiclass case

Let the model output logits $z_1, \dots, z_C$. Convert them to probabilities with softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}.$$

Feature maps and receptive fields

Feature maps

Each convolution layer produces multiple channels:

$$X \in \mathbb{R}^{H \times W \times C} \longrightarrow Y \in \mathbb{R}^{H' \times W' \times K}.$$

Receptive field

The receptive field of a unit is the region of the input image that can influence that unit.

Why depth helps

Stacking layers enlarges the effective receptive field. Early units see local edges; deeper units can respond to larger object parts.

Practical implication

Resolution, kernel size, stride, and pooling all affect what spatial scale the model can capture.

When the loss and the metric differ

Why this happens

Accuracy, F1, AUC, and calibration error are useful metrics, but they are not always convenient optimization objectives.

Practical consequence

You may train with cross-entropy, then select and report using validation AUC, F1, or calibration metrics.

What is regularization?

Goal

Regularization reduces overfitting and improves generalization.

- A model can fit training data too well and still fail on new data.
- Regularization encodes preferences for simpler or more robust solutions.
- In small-data settings, regularization is often essential rather than optional.

Common regularization tools

Parameter-based

- Weight decay / L_2 penalty
- Smaller models
- Early stopping

Data and activation based

- Dropout
- Data augmentation
- Batch normalization as a stabilizer

Weight decay and dropout

Weight decay

Add a penalty on large weights:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \|W\|_2^2.$$

This discourages overly complex parameter values.

Dropout

Randomly zero some activations during training so the model does not depend too heavily on any single path.

Use with judgment

On modern pretrained CNN backbones, strong dropout is not always necessary. Validation performance should decide.

Batch normalization

Core idea

Normalize activations within a mini-batch, then learn a scale and shift:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta.$$

Parameters

- μ_B : batch mean
- σ_B^2 : batch variance
- γ, β : learned affine parameters

Why it helps

- Makes optimization more stable.
- Allows larger learning rates in many cases.
- Reduces sensitivity to activation scale.

Practical note

Batch normalization behaves differently in training and evaluation mode, so the model state must be set correctly during validation and testing.

Data augmentation

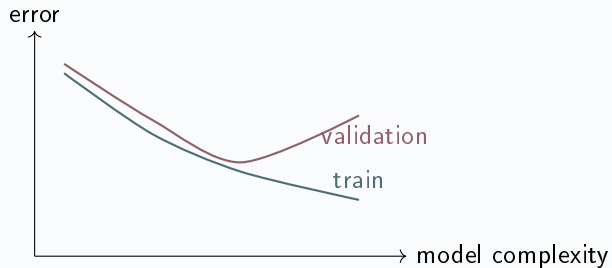
- Random crop
- Horizontal flip
- Color jitter
- Small rotation
- Random resized crop

Key constraint

Augmentations must respect the semantics of the task.

- For leaf classification, moderate flips and crops can be sensible.
- For medical data, colour changes or aggressive crops may destroy relevant evidence.
- For matching tasks, paired transformations must preserve geometric consistency.

Overfitting and underfitting



Interpretation

Training error usually falls as complexity increases. Validation error often falls first, then rises once overfitting begins.

What is model selection?

Definition

Model selection means choosing hyperparameters or architectures using validation data rather than the test set.

- Learning rate
- Weight decay
- Batch size
- Number of epochs
- Which layers to freeze
- Augmentation strength

Train, validation, test

Train

Fit parameters.

Validation

Choose settings and stop training.

Test

Final unbiased estimate.

Strict rule

Do not tune hyperparameters on the test set. Once the test set influences decisions, it is no longer a proper test set.

Early stopping

Idea

Stop training when validation performance no longer improves enough to justify more updates.

Validation-loss version

Let $L_{\text{val}}^{(t)}$ be the validation loss at epoch t and

$$t^* = \arg \min_t L_{\text{val}}^{(t)}.$$

Early stopping returns the checkpoint from epoch t^* , not the final epoch.

Patience rule

Stop if there has been no improvement for p epochs:

$$L_{\text{val}}^{(t)} \geq \min_{s \leq t-p} L_{\text{val}}^{(s)} \quad \text{for } p \text{ consecutive epochs.}$$

How to compare models fairly

- Use the same data split.
- Use the same evaluation metric.
- Report parameter count or training cost when relevant.
- Keep preprocessing consistent unless the method requires otherwise.
- Repeat over multiple seeds for unstable setups.

For this course

A classical baseline and a deep baseline should be compared under matched experimental conditions as much as possible.

Overview: what happens where?

Model design stage

- Choose network type: MLP or CNN.
- Choose depth, channels, kernel sizes.
- Decide whether to use BatchNorm, pooling, dropout.

Training stage

- Forward pass produces predictions.
- Loss function measures prediction error.
- Backpropagation computes gradients.
- Optimizer updates parameters.

Generalization stage

- Regularization reduces overfitting.
- Data augmentation acts on training inputs.
- Weight decay and dropout act during training.
- Early stopping uses validation curves.

Selection and reporting stage

- Validation chooses hyperparameters and best epoch.
- Test set is used only once at the end.
- Fair comparison needs matched splits and metrics.

Signs your setup is broken

- 1 Training loss does not decrease at all.
- 2 Validation performance is unrealistically perfect.
- 3 Changing the model has no effect on the result.
- 4 Best epoch is never saved.
- 5 Results depend entirely on one random seed.

First debugging move

Try to overfit a tiny subset such as 10 examples. If the model cannot do that, the pipeline is probably incorrect.